Unidad 3.- Diseño orientado a objetos: Diagramas de Clases

EJERCICIO 1 (0.5 ptos)

a) Se han identificado los conceptos básicos de la programación orientada a objetos. 5%

Explica los fundamentos del paradigma de Programación Orientada a Objetos (POO), definiendo sus **conceptos esenciales** y cuáles son sus **principales ventajas.**

EJERCICIO 2 (4 ptos)

b) Se han utilizado herramientas para la elaboración de diagramas de clases.15%

d) Se han trazado diagramas de clases a partir de las especificaciones de las mismas.25%

Diseña el diagrama de clases para el siguiente sistema que representa una **BIBLIOTECA**, para ello, se debe utilizar la herramienta **DIA**.

Se incluirá la **imagen JPG** del diagrama de clases generado en **DIA**.

A continuación, se indican las diferentes **CLASES** con sus **atributos** y **métodos**:

1) Crea una clase con el nombre **libro** que tenga:
    a) Atributos: ISBN (tipo int), temática (tipo string), editorial (tipo string), y año (tipo int). Los libros tienen copias.

2) Crea una clase con el nombre **copia** que tenga:
    a) Atributos: id_copia (tipo int) y estado (tipo string).

3) Crea una clase con el nombre **autor** que tenga:
    a) Atributos: id_autor (tipo int), nombre(tipo string), nacionalidad (tipo string) y fecha de nacimiento (tipo int).

4) Crea una clase con el nombre **sociolector** que tenga:
    a) Atributos: num_socio (tipo int), nombre (tipo string), dirección (tipo string), móvil (tipo int), totalMulta (tipo int), numPréstamos (tipo int).
    b) Métodos:
        i) **prestar** recibe el id_copia (tipo int) y fAd (tipo date) y devuelve un booleano.
        ii) **devolver** recibe el id_copia (tipo int) y fechaActual (tipo date) y no devuelve nada.

5) Crea una clase con el nombre **multa** que tenga:
    a) Atributos: mInit (tipo date) y mFin (tipo date)

6) Crea una clase con el nombre **prestamo** que tenga:
    a) Atributos: pinit (tipo date) y pfin (tipo date)

7) Establece una relación de **Asociación** entre **libro** y **copia** y establece la cardinalidad según lo siguiente:
    a) Un libro puede tener de una a muchas copias. Toda copia solo pertenece a un libro.

8) Establece una relación de **Asociación** entre **autor** y **libro** y establece la cardinalidad según lo siguiente:
    a) Un autor puede escribir de uno a muchos libros y todo libro pertenece a un único autor.

9) Establece una relación de **Asociación** entre **sociolector** y **multa** y establece la cardinalidad según lo siguiente:
    a) Un sociolector puede tener de cero a muchas multas y una multa pertenece a un único sociolector.

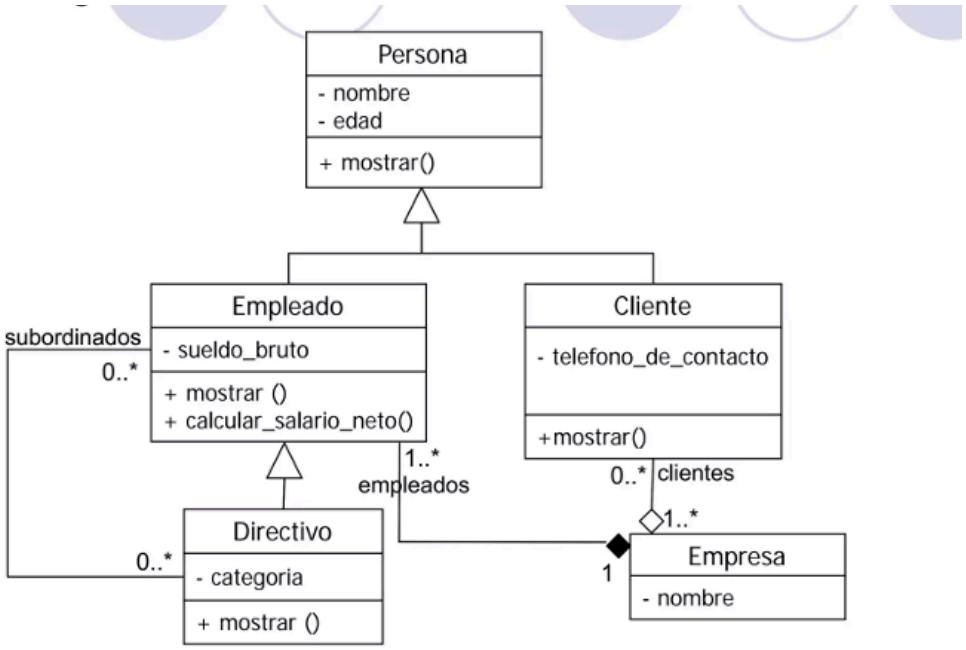
10) Establece **atributos a la relación** entre **copia** y **sociolector** y establece la cardinalidad según lo siguiente:
    a) Los lectores pueden tener de cero a tres copias en **préstamo** y toda copia puede estar en **préstamo** con cero o un sociolector.

EJERCICIO 3 (4 ptos)

c) Se ha interpretado el significado de diagramas de clases.15%
e) Se ha generado código a partir de un diagrama de clases. 25%

A partir del siguiente diagrama de clases, genera el código java correspondiente.

Para ello, se debe utilizar el IDE: **NETBEANS** y debe crear un nuevo package con todas las clases detectadas. Adjunta la copia del código correspondiente. **NO se admiten capturas de pantalla.**



EJERCICIO 4 (1.5 ptos)

f) Se ha generado un diagrama de clases mediante ingeniería inversa.15%

A partir del siguiente código java genera el diagrama de clases UML con **NETBEANS**, para ello haz uso del plugin **PlantUML-NB**.

Adjunta la captura de pantalla que lo acredite.

La utilización de este plugin queda relegada a este ejercicio y solo para generar el diagrama correspondiente. **Cualquier otro uso de PlantUML-NB invalidará el examen con cero.**

```
public class Empleado {
    protected String nombre;
    protected double salario;

    public Empleado(String nombre, double salario) {
        this.nombre = nombre;
        this.salario = salario;
    }

    public void mostrarDatos() {
        System.out.println("Nombre: " + nombre + ", Salario: " + salario);
    }
}

public class Gerente extends Empleado {
    private String departamento;

    public Gerente(String nombre, double salario, String departamento) {
        super(nombre, salario);
        this.departamento = departamento;
    }

    @Override
    public void mostrarDatos() {
        super.mostrarDatos();
        System.out.println("Departamento: " + departamento);
    }
}
```

```java
public class Empresa {
    private String nombreEmpresa;
    private Gerente director; // Asociación

    public Empresa(String nombreEmpresa, Gerente director) {
        this.nombreEmpresa = nombreEmpresa;
        this.director = director;
    }

    public void imprimirNomina() {
        System.out.println("Empresa: " + nombreEmpresa);
        director.mostrarDatos();
    }
}
```

FORMATO DE ENTREGA

Se realizarán dos entregas en AV:

1. Fichero **PDF con el Nombre_Apellido** del estudiante, con cada EJERCICIO 1, etc,,, SIN ENUNCIADO y con la respuesta correspondiente. Todos los ejercicios irán ordenados, si no se responde, debe aparecer en el listado el ejercicio y su número.
2. **Carpeta comprimida Nombre_Apellido** del estudiante con **Código Java** con cada clase generada en Netbeans.

SE CALIFICARÁ SOBRE EL FICHERO.PDF ENTREGADO.